Hibernate ID Generation Strategies

Every entity needs a unique primary key. The **@GeneratedValue** annotation tells Hibernate how the ID should be created. The best strategy depends on the database, performance needs, and how much control the application requires.

1. GenerationType.AUTO

What is it?

AUTO lets the persistence provider choose an appropriate ID strategy. For a numeric ID, Hibernate selects a strategy that works with the current database, such as a sequence, identity column, or generator table.

When to use it?

Use it when database portability is more important than controlling the exact generation method. It is convenient for small applications and prototypes. It is less suitable when the project depends on a specific SQL structure or predictable insert performance.

Example

```
@Id
@GeneratedValue(strategy = GenerationType.AUTO)
private Long id;
```

Database

It can work with most relational databases, including PostgreSQL, MySQL, Oracle, SQL Server, and H2. The actual strategy may be different on each database.

2. GenerationType.IDENTITY

What is it?

IDENTITY uses an identity or auto-increment column. The database creates the ID when the INSERT statement is executed, and Hibernate reads the generated value afterward.

When to use it?

Use it when the database already has a simple auto-increment feature and the application does not need IDs before insertion. It is easy to understand and is a common choice for basic CRUD applications. A limitation is that Hibernate usually must execute the insert immediately to obtain the ID, which can reduce insert batching.

Example

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

Database

Good choices include MySQL, MariaDB, SQL Server, H2, and databases that support SQL identity columns. The Teacher entity in this project uses IDENTITY.

3. GenerationType.SEQUENCE

What is it?

SEQUENCE obtains IDs from a database sequence. A sequence is an independent database object that returns new numeric values. Hibernate can request an ID before inserting the entity.

When to use it?

Use it when the database supports sequences, especially when the application performs many inserts. Hibernate can reserve a block of IDs through **allocationSize**, which reduces database calls and works well with batching. Gaps between IDs are normal and should not be treated as errors.

Example

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE,
                generator = "teacher_seq")
@SequenceGenerator(name = "teacher_seq",
                   sequenceName = "teacher_sequence",
                   allocationSize = 10)
private Long id;
```

Database

It is a strong choice for PostgreSQL, Oracle, DB2, and H2. MySQL does not provide normal database sequences, so SEQUENCE is not the usual choice there.

4. GenerationType.TABLE

What is it?

TABLE stores the next ID value in a separate database table. Hibernate reads and updates a row in that table whenever it needs a new range of IDs. This simulates a database sequence using ordinary tables.

When to use it?

Use it when a project needs a portable generator but the database does not support sequences or identity columns in the required way. It can also be useful in some legacy systems. It is normally the slowest strategy because generating IDs requires extra table access, locking, and updates.

Example

```
@Id
@GeneratedValue(strategy = GenerationType.TABLE,
                generator = "teacher_table_gen")
@TableGenerator(name = "teacher_table_gen",
                table = "id_generator",
                pkColumnName = "entity_name",
                valueColumnName = "next_id",
                pkColumnValue = "teacher",
                allocationSize = 10)
private Long id;
```

Database

It works with almost any relational database, including MySQL, PostgreSQL, Oracle, SQL Server, and H2, because it only needs a normal table.

Simple Comparison

Strategy	How the ID is created	When to choose it	Suitable databases
AUTO	Hibernate chooses a compatible strategy.	Portability, prototypes, and cases where the exact method is not important.	Most relational databases.
IDENTITY	The database creates the ID during INSERT using an identity or auto-increment column.	Simple CRUD systems and databases with strong auto-increment support.	MySQL, MariaDB, SQL Server, H2.
SEQUENCE	Hibernate requests values from a database sequence, often in blocks.	High insert performance, batching, and databases with sequence support.	PostgreSQL, Oracle, DB2, H2.
TABLE	A separate table stores and updates the next available ID.	Legacy or portable solutions when other database generators are unsuitable.	Almost any relational database.

How to Choose

Choose AUTO when Hibernate may decide and portability is the main goal.

Choose IDENTITY for a simple auto-increment design, especially with MySQL.

Choose SEQUENCE when the database supports sequences and insert performance matters.

Choose TABLE only when a table-based portable generator is actually needed, because it adds more database work.

Final Idea

The strategies do not only produce different SQL. They also change when the ID becomes available and how efficiently Hibernate can insert many records. IDENTITY is simple, SEQUENCE is usually better for batching, TABLE is the most portable but often the least efficient, and AUTO gives the decision to Hibernate.